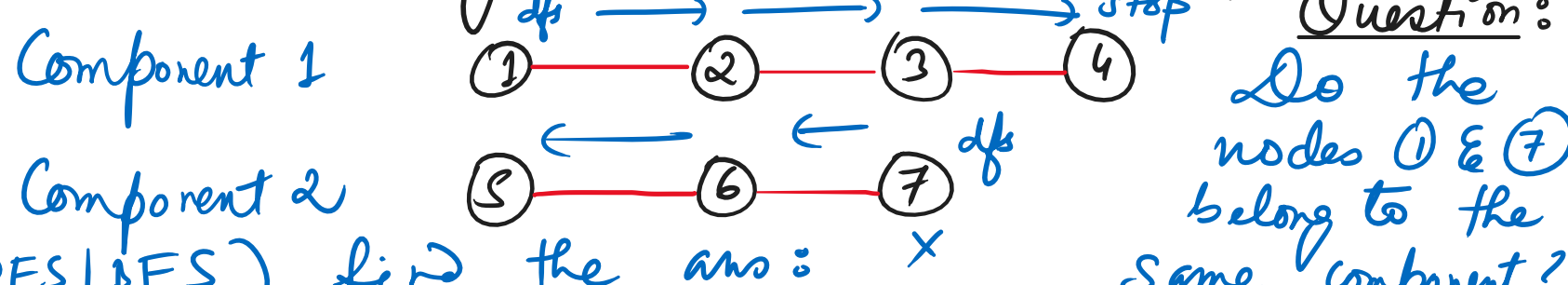


* Disjoint Set Data Structure Concept :-

Let's consider a graph with 2 Components:



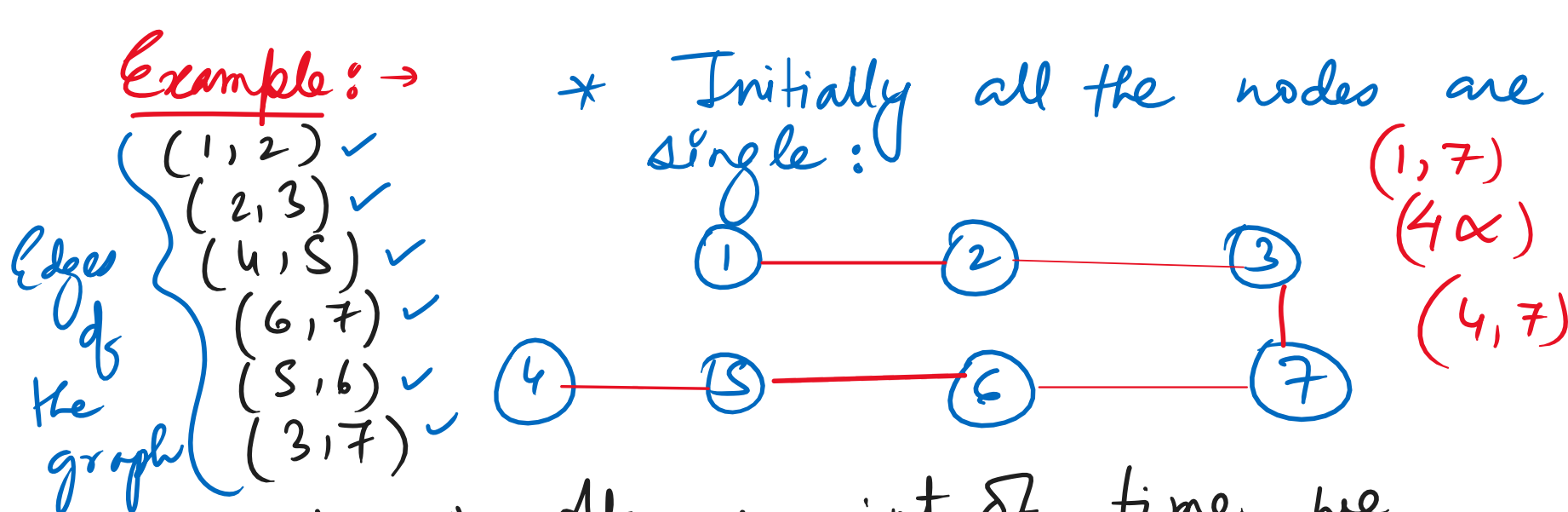
Question: Do the nodes 1 & 7 belong to the same component?
(DFS/BFS) find the ans: $\times$

Time Complexity = $O(N+E)$ Linear TC [No]

Disjoint Set Use Case?

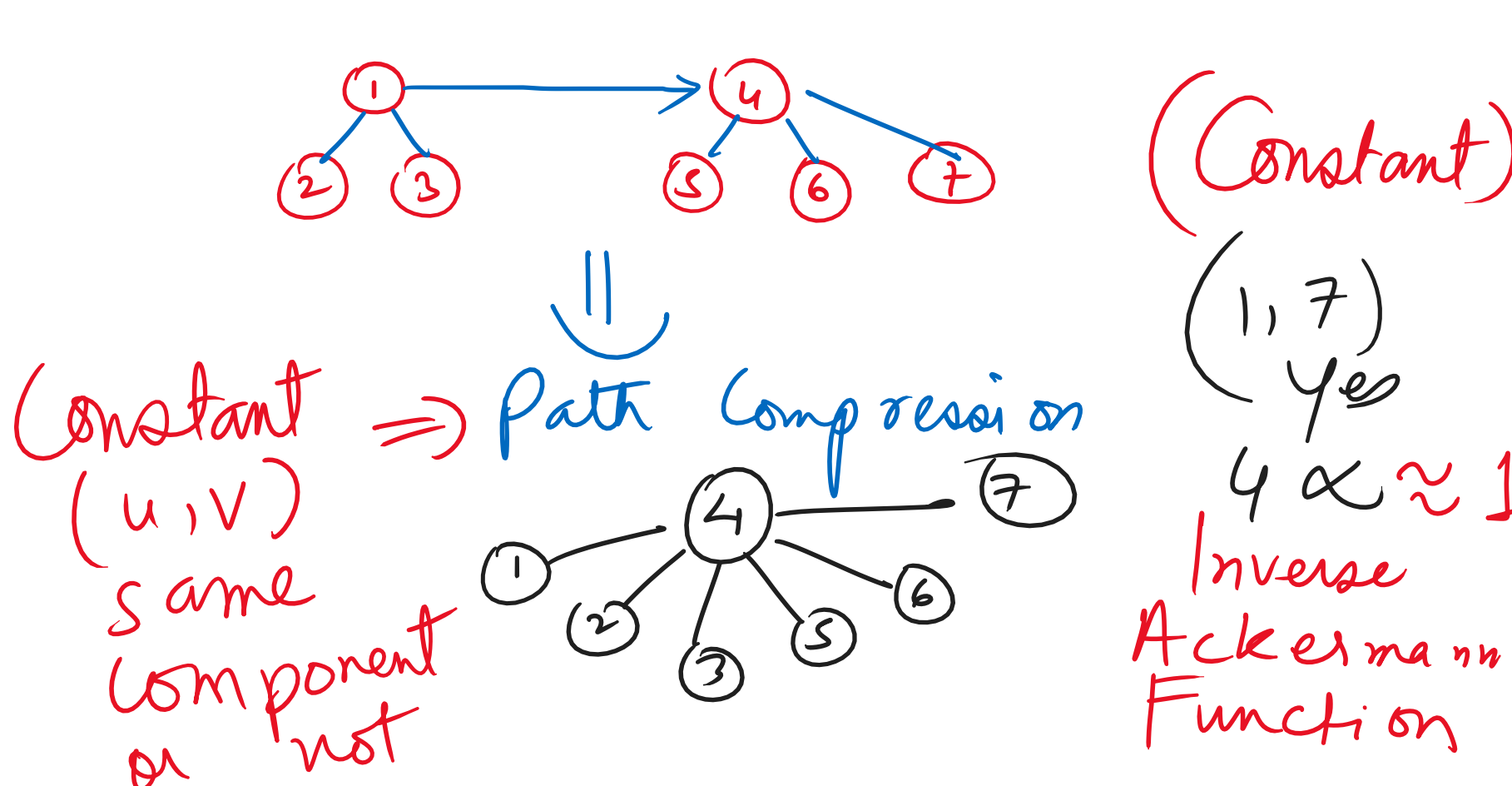
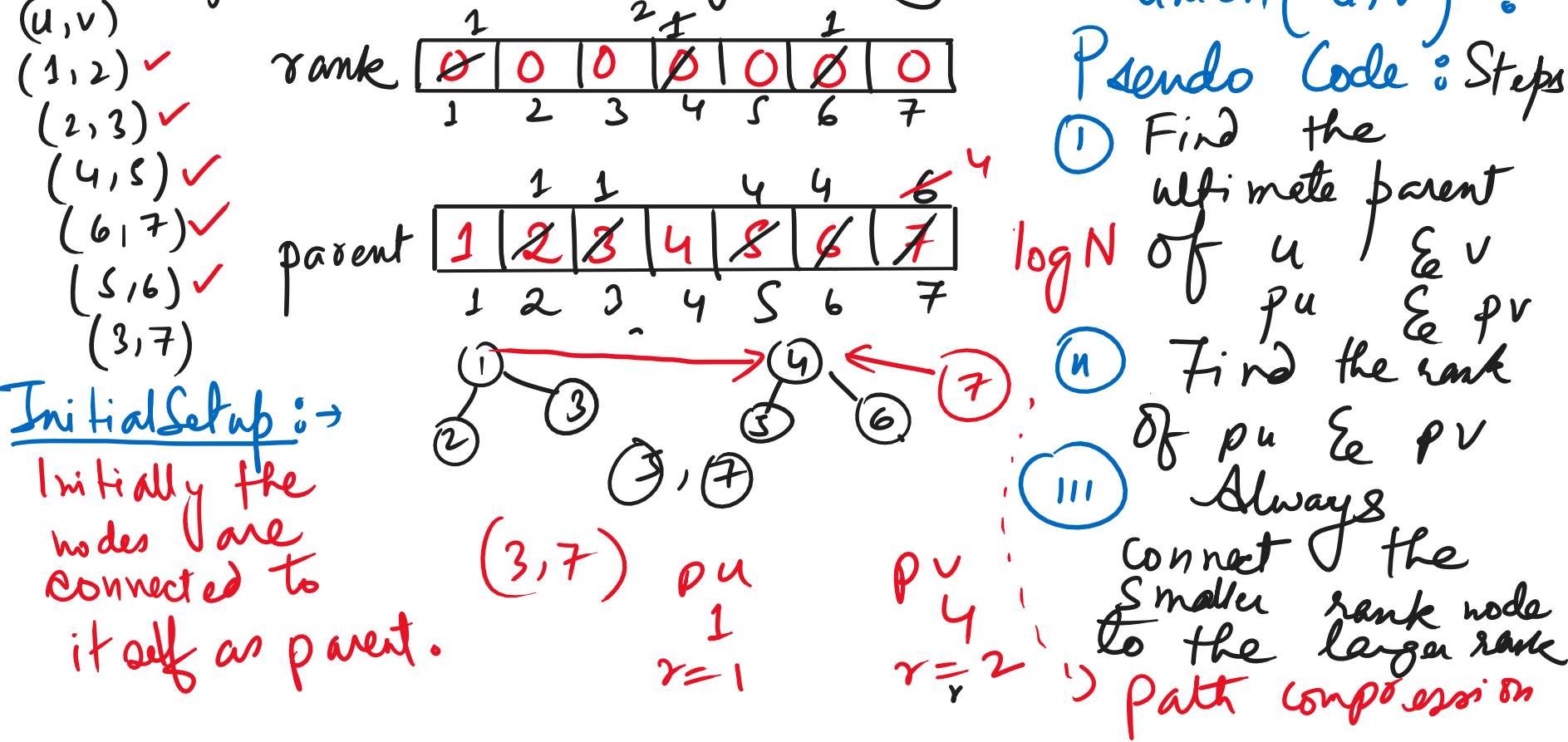
Check whether two nodes lie in the same component, in constant time (4x)
specifically when the graph is changing dynamically [Dynamic graphs]

Example :-

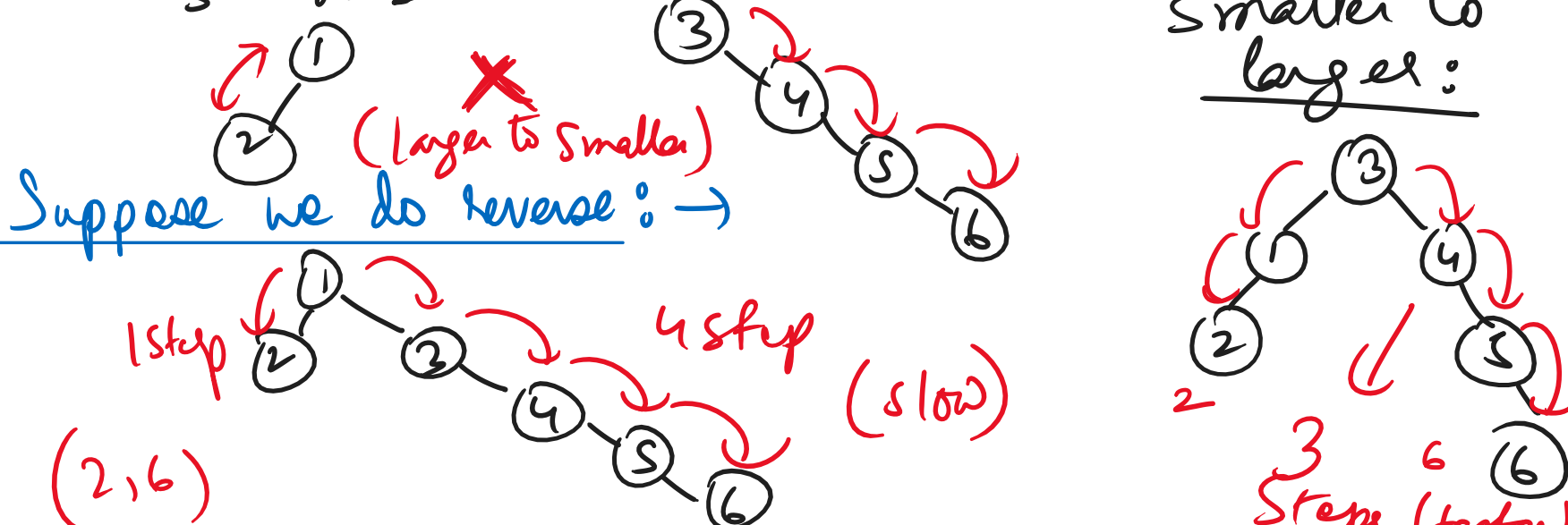


Criteria: At any point of time we should be able to answer where any two nodes belong to the same component in Constant Time (Disjoint Set)

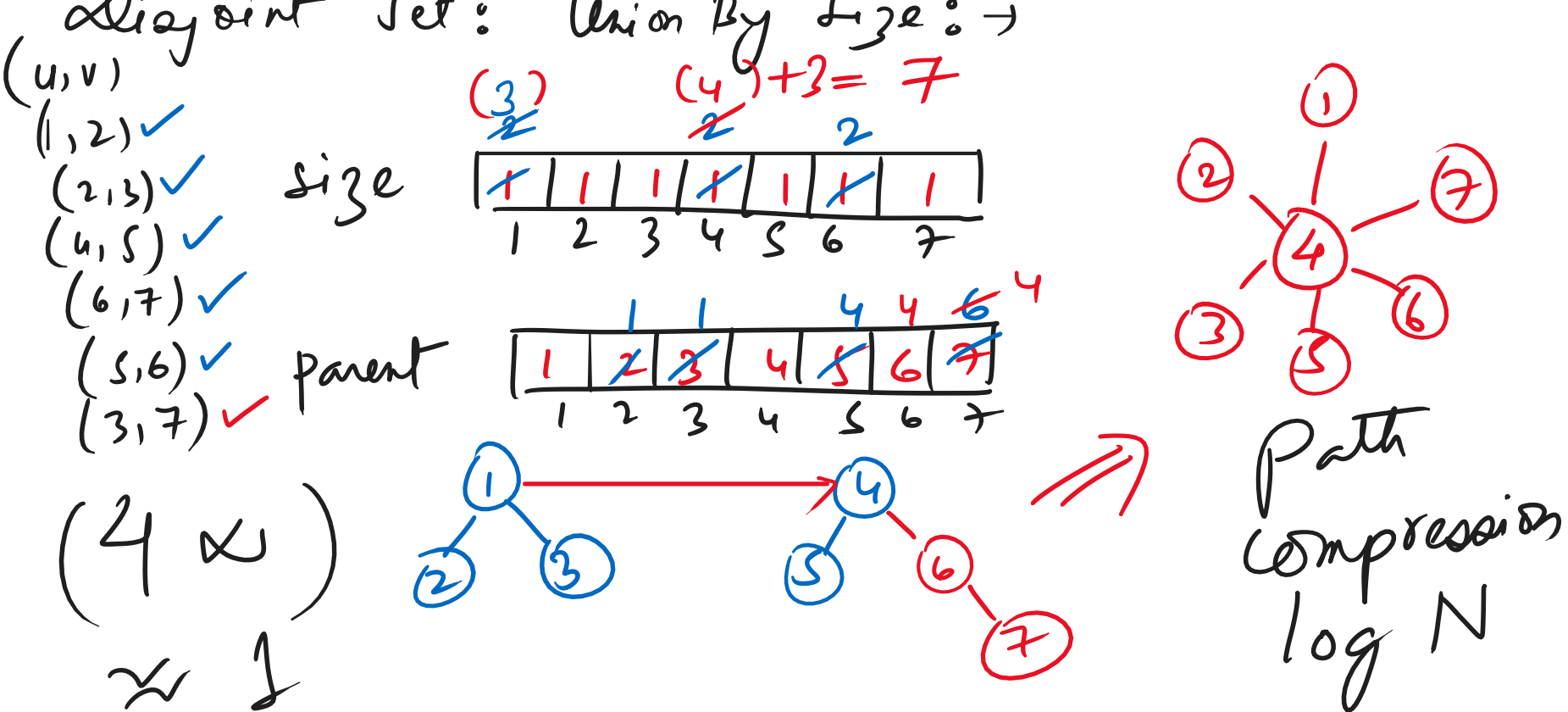
Disjoint Set [Union by Rank] :-



* Why do we even have to connect a smaller rank node to a larger rank node?



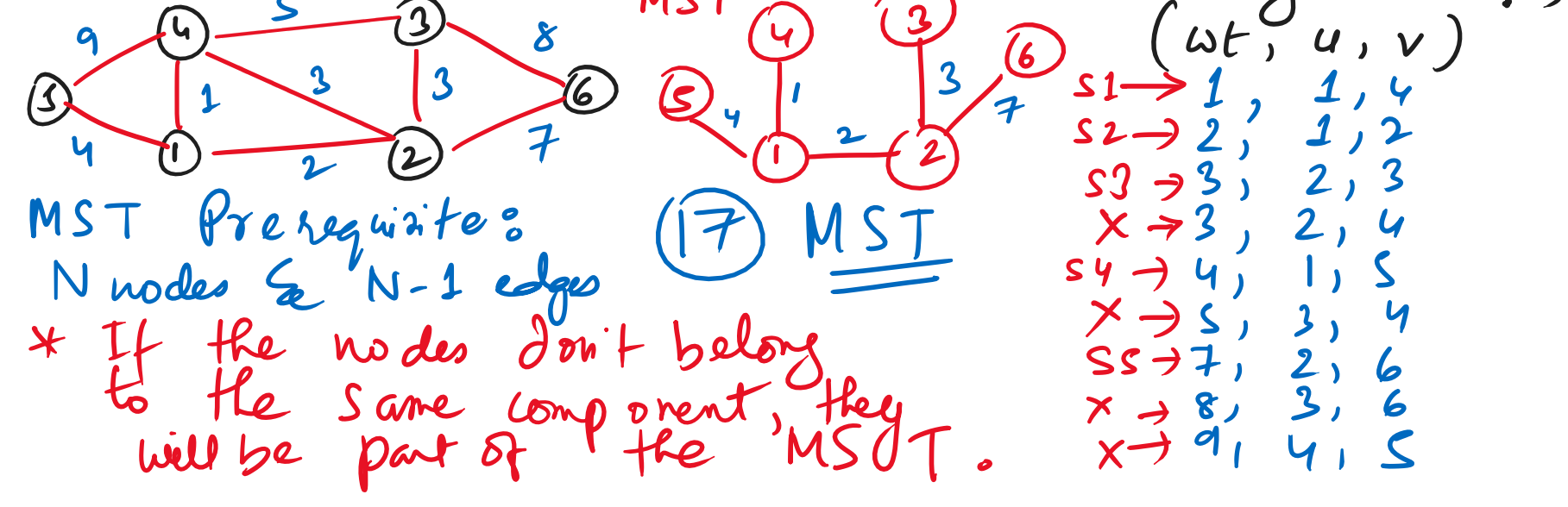
Disjoint Set: Union By Size :-



Kruskal's Algorithm (Minimum Spanning Tree)

[Joseph Kruskal] [American Mathematician]

* (Disjoint Set Data Structure) * Step 1: Sort the edges in ascending order :-



Dynamic Programming: (DP)

"Those who forget the past, are forced to repeat it!"

* Defn :- Solution of smaller overlapping subproblems to find the solution of a bigger problem.

* It can be performed on 1D, 2D arrays.

* There are 4 steps in DP :-

- * (1) Recursion
- * (2) Memoisation
- * (3) Tabulation
- * (4) Space Optimisation

(Fibonacci Series :- n^{th} Fib Number)

0, 1, 1, 2, 3, 5, 8, 13, 21, ... (n $\rightarrow$ n-1) Recursion

0 1 2 3 4 5 6 7 8 $\rightarrow$ previous

$f(2) = f(1) + f(0)$

$f(3) = f(2) + f(1)$

$f(n) = f(n-1) + f(n-2)$

$f(0) = 0, f(1) = 1$

5th fibonacci number $\rightarrow$ Recursion Tree

* When you do all recursive calls $\rightarrow$ normal recursion

* When you know the answer & don't call extra

$\rightarrow$ Memoisation

* Store known value [Tabulation]

* Optimize space $\rightarrow$ S'O

overlapping subproblem

$R \rightarrow 2^n$

$M \rightarrow n^2$

* This won't happen

this won't happen

Memoisation